

Power BI Insurance Analytics

End-to-End Project — Star Schema + 40 DAX KPIs

Domain	Insurance (Health, Life, Motor, Home, Travel)
Project Type	Power BI Business Intelligence Dashboard
Data Model	Star Schema — 3 Dimension Tables + 1 Fact Table + Date Table
Total Rows	~2,515 rows across 4 workbooks
DAX Functions	40 unique KPIs covering all major DAX categories
Dashboard Pages	5 pages — Overview, Claims, Policy, Agent, Customer
Skill Level	Intermediate to Advanced (Interview Ready)

This project is designed to be explained in student interviews and demonstrates real-world Power BI skills including data modelling, DAX, dashboard design, and business storytelling.

1. Project Overview & Business Context

InsureVision Analytics is a Power BI project built for a fictional Insurance company — **SafeGuard Insurance Pvt. Ltd.** — operating across India. The company sells multiple insurance products (Health, Life, Motor, Home, Travel, Term Life, Endowment, ULIP) through various channels (Agents, Online, Banks, Brokers). The management team requires a comprehensive analytics dashboard to monitor policy performance, claims efficiency, agent productivity, customer segmentation, and overall financial health of the portfolio.

1.1 Business Problem Statement

The company faces the following key challenges that this Power BI project will address:

- High claim rejection rates causing customer dissatisfaction and regulatory scrutiny.
- Inability to identify which policy types generate maximum premium revenue.
- Poor visibility into agent performance across different regions.
- No real-time KPI monitoring for claim settlement ratio and turnaround time.
- Difficulty in identifying high-value customers (Premium Segment) and their policy mix.
- No fraud detection mechanism across the claims portfolio.
- Seasonal trends in claim filing not being captured for planning purposes.
- Customer retention metrics not measured leading to high policy lapse rates.

1.2 Project Goals

- ✓ Build a Star Schema data model with properly defined relationships.
- ✓ Create a dynamic Date Table using CALENDAR() DAX function.
- ✓ Design 40 KPIs using different DAX functions covering all business areas.
- ✓ Build 5 interactive dashboard pages for different stakeholder groups.
- ✓ Enable drill-through, bookmarks, and dynamic titles in the report.
- ✓ Implement Row Level Security (RLS) concept for agent-level data access.

2. Data Model — Star Schema

The project uses a **Star Schema** — one central Fact Table surrounded by Dimension Tables. This is the most recommended model for Power BI for performance and DAX simplicity.

2.1 Star Schema Diagram (Text Representation)

Table	Type	Primary Key	Rows	Connects To
DIM_Customer	Dimension	CustomerID	500	FACT_Claims (CustomerID)
DIM_Policy	Dimension	PolicyID	800	FACT_Claims (PolicyID, CustomerID)
DIM_Agent	Dimension	AgentID	15	DIM_Policy (AgentID)
DIM_Date	Date Table	Date	Dynamic	FACT_Claims (ClaimDate)
FACT_Claims	Fact Table	ClaimID	1200	All Dimension Tables

2.2 Relationship Details (No Ambiguity)

All relationships are **One-to-Many (*:1)** from Fact to Dimension with **Single direction cross-filter**. This ensures a clean star schema with no circular dependencies.

From Table → Column	To Table → Column	Cardinality	Cross Filter	Active
FACT_Claims → CustomerID	DIM_Customer → CustomerID	Many to One (*:1)	Single	Yes
FACT_Claims → PolicyID	DIM_Policy → PolicyID	Many to One (*:1)	Single	Yes
DIM_Policy → AgentID	DIM_Agent → AgentID	Many to One (*:1)	Single	Yes
FACT_Claims → ClaimDate	DIM_Date → Date	Many to One (*:1)	Single	Yes

Important: DIM_Policy has TWO columns (PolicyID and CustomerID) that connect to FACT_Claims. Only the PolicyID → PolicyID relationship should be ACTIVE. CustomerID → CustomerID in DIM_Policy to FACT_Claims should be set as INACTIVE to avoid ambiguity. Use USERELATIONSHIP() in DAX when needed.

3. Dataset Descriptions

3.1 DIM_Customer.xlsx — 500 Rows

This is the Customer Dimension table. Each row represents one unique customer. CustomerID is the Primary Key and must be unique. It connects to FACT_Claims via CustomerID.

Column	Data Type	Sample Value	Description
CustomerID	Text	CUST0001	Unique Customer ID — Primary Key
CustomerName	Text	Rajesh Kumar	Full name of customer
Age	Number	35	Customer age in years
Gender	Text	Male/Female	Customer gender
DateOfBirth	Date	1989-03-15	Date of birth (YYYY-MM-DD)
Occupation	Text	Salaried	Employment type
AnnualIncome	Number	600000	Annual income in INR
State	Text	Maharashtra	State of residence
City	Text	Mumbai	City of residence
PinCode	Text	400001	6-digit pin code
Email	Text	rajesh@email.com	Email address
PhoneNumber	Text	9812345678	10-digit mobile number
CustomerSegment	Text	Premium/Standard/Basic	Segment based on income
MemberSince	Date	2018-06-01	Date of first policy
MaritalStatus	Text	Married	Marital status
Dependents	Number	2	Number of dependents

3.2 DIM_Policy.xlsx — 800 Rows

This is the Policy Dimension table. Each row represents one unique insurance policy. PolicyID is the Primary Key. Multiple policies can belong to the same customer (one customer → many policies).

Column	Data Type	Sample Value	Description
PolicyID	Text	POL00001	Unique Policy ID — Primary Key
CustomerID	Text	CUST0001	Foreign Key linking to DIM_Customer
PolicyType	Text	Health	Type: Health/Life/Motor/Home/Travel/Term Life/Endowment/ULIP
PolicyName	Text	Health Shield Plan	Product name
CoverageAmount	Number	500000	Sum assured in INR

Column	Data Type	Sample Value	Description
AnnualPremium	Number	12000	Annual premium in INR
PaymentFrequency	Text	Monthly	Monthly/Quarterly/Half-Yearly/Annually
PolicyStartDate	Date	2020-01-01	Policy inception date
PolicyEndDate	Date	2030-01-01	Policy maturity/expiry date
PolicyDuration_Years	Number	10	Policy tenure in years
PolicyStatus	Text	Active	Active/Lapsed/Expired/Surrendered
AgentID	Text	AGT001	Foreign Key linking to DIM_Agent
Channel	Text	Agent	Sales channel
RiderIncluded	Text	Critical Illness	Optional add-on rider
NomineeRelation	Text	Spouse	Relationship of nominee
PremiumPaymentMode	Text	Auto-Debit	Mode of premium payment

3.3 FACT_Claims.xlsx — 1,200 Rows (Main Fact Table)

This is the central Fact Table containing all insurance claims. Each row represents one unique claim. It contains measures (ClaimAmount, ApprovedAmount, ProcessingDays) and foreign keys to all dimension tables. This is the table on which most DAX measures are written.

Column	Data Type	Sample Value	Description
ClaimID	Text	CLM000001	Unique Claim ID — Primary Key
PolicyID	Text	POL00001	Foreign Key → DIM_Policy (ACTIVE relationship)
CustomerID	Text	CUST0001	Foreign Key → DIM_Customer
ClaimDate	Date	2022-05-10	Date claim was filed — links to DIM_Date
ClaimType	Text	Hospitalization	Nature of claim
ClaimAmount	Number	150000	Total claim amount filed in INR
ApprovedAmount	Number	150000	Amount approved by insurer (0 if rejected)
ClaimStatus	Text	Approved	Approved/Rejected/Pending/Under Review/Partially Approved
RejectionReason	Text	Pre-existing Condition	Reason if rejected (blank if approved)
ProcessingDays	Number	12	Days taken to process the claim
HospitalName	Text	Apollo Hospital	Hospital/service provider name
DoctorName	Text	Dr. Ramesh	Treating doctor
ClaimMonth	Number	5	Month number (1-12) for easy filtering
ClaimYear	Number	2022	Year of claim
SurveyorID	Text	SRV001	Claims surveyor ID
FraudFlag	Text	No	Yes/No — potential fraud indicator
ClaimCategory	Text	High	High/Medium/Low based on claim amount

3.4 DIM_Agent.xlsx — 15 Rows

This is the Agent Dimension table. Contains details of 15 insurance agents. AgentID is the Primary Key and connects to DIM_Policy via AgentID.

Column	Data Type	Sample Value	Description
AgentID	Text	AGT001	Unique Agent ID — Primary Key
AgentName	Text	Rajesh Kumar	Full name of agent
Region	Text	North	Sales region: North/South/East/West/Central
State	Text	Maharashtra	State where agent operates

Column	Data Type	Sample Value	Description
City	Text	Mumbai	City of agent
JoiningDate	Date	2015-03-01	Date agent joined company
AgentType	Text	Field Agent	Field Agent/Tele Agent/Broker/Online Agent
Qualification	Text	MBA	Educational qualification
ExperienceYears	Number	9	Total years of experience
Target_Annual	Number	2000000	Annual premium collection target in INR
ManagerID	Text	MGR001	Reporting manager ID
AgentStatus	Text	Active	Active/Inactive
SpecializationProduct	Text	Health	Primary product specialty
LicenseNumber	Text	LIC234567	IRDAI license number

4. Date Table — Created Using CALENDAR() in Power BI

The Date Table (DIM_Date) must be created inside Power BI Desktop using DAX. Do NOT import it from Excel. Use the following DAX code in 'Modeling → New Table':

```
DIM_Date = ADDCOLUMNS( CALENDAR(DATE(2019,1,1), DATE(2024,12,31)), "Year", YEAR([Date]),
"Month", MONTH([Date]), "MonthName", FORMAT([Date], "MMMM"), "ShortMonth", FORMAT([Date],
"MMM"), "Quarter", QUARTER([Date]), "QuarterName", "Q" & QUARTER([Date]), "YearQuarter",
YEAR([Date]) & "-Q" & QUARTER([Date]), "YearMonth", FORMAT([Date], "YYYY-MM"), "WeekDay",
WEEKDAY([Date], 2), "DayName", FORMAT([Date], "DDDD"), "DayOfMonth", DAY([Date]),
"IsWeekend", IF(WEEKDAY([Date],2) >= 6, "Weekend", "Weekday"), "FiscalYear",
IF(MONTH([Date]) >= 4, "FY" & YEAR([Date]) & "-" & RIGHT(YEAR([Date])+1, 2), "FY" &
YEAR([Date])-1 & "-" & RIGHT(YEAR([Date]), 2)), "FiscalQuarter", IF(MONTH([Date]) >= 4,
"Q" & INT((MONTH([Date])-4)/3)+1, "Q" & INT((MONTH([Date])+8)/3)+1) )
```

Step: After creating the table, go to **Modeling tab** → **Mark as Date Table** and select the 'Date' column. This enables Time Intelligence functions like TOTALYTD(), SAMEPERIODLASTYEAR() etc.

5. 40 DAX KPI Measures — Complete Guide

Below are 40 DAX measures using 40 different and important DAX functions. These cover Aggregation, Time Intelligence, Filter, Statistical, Logical, Text, Iterator, and Table functions.

1. Total Claims Filed (Aggregation)

Business Use: Total number of claims filed in selected period

```
Total_Claims_Filed = COUNT(FACT_Claims[ClaimID])
```

2. Total Claim Amount (Aggregation)

Business Use: Sum of all claim amounts filed

```
Total_Claim_Amount = SUM(FACT_Claims[ClaimAmount])
```

3. Total Approved Amount (Aggregation)

Business Use: Sum of all approved claim amounts

```
Total_Approved_Amount = SUM(FACT_Claims[ApprovedAmount])
```

4. Total Annual Premium (Aggregation)

Business Use: Total premium collected from all policies

```
Total_Annual_Premium = SUM(DIM_Policy[AnnualPremium])
```

5. Average Claim Amount (Aggregation)

Business Use: Average value of each claim filed

```
Avg_Claim_Amount = AVERAGE(FACT_Claims[ClaimAmount])
```

6. Max Claim Amount (Aggregation)

Business Use: Highest single claim ever filed

```
Max_Claim_Amount = MAX(FACT_Claims[ClaimAmount])
```

7. Min Claim Amount (Aggregation)

Business Use: Lowest single claim filed

```
Min_Claim_Amount = MIN(FACT_Claims[ClaimAmount])
```

8. Total Active Policies (Aggregation)

Business Use: Count of all currently active policies

```
Total_Active_Policies = CALCULATE( COUNTROWS(DIM_Policy), DIM_Policy[PolicyStatus] = "Active" )
```

9. Claim Settlement Ratio (Ratio / %)

Business Use: % of claims approved out of total filed. Industry benchmark: >90%

```
Claim_Settlement_Ratio = DIVIDE( CALCULATE(COUNT(FACT_Claims[ClaimID]), FACT_Claims[ClaimStatus] = "Approved"), COUNT(FACT_Claims[ClaimID]), 0 ) * 100
```

10. Claim Rejection Rate (Ratio / %)

Business Use: % of claims rejected — key risk/compliance KPI

```
Claim_Rejection_Rate = DIVIDE( CALCULATE(COUNT(FACT_Claims[ClaimID]),  
FACT_Claims[ClaimStatus] = "Rejected"), COUNT(FACT_Claims[ClaimID]), 0 ) * 100
```

11. Loss Ratio (Ratio / %)

Business Use: Claims paid vs premium collected. Below 70% is considered healthy

```
Loss_Ratio = DIVIDE( [Total_Approved_Amount], [Total_Annual_Premium], 0 ) * 100
```

12. Avg Processing Days (Iterator)

Business Use: Average number of days taken to process a claim using row-by-row iteration

```
Avg_Processing_Days = AVERAGEX( FACT_Claims, FACT_Claims[ProcessingDays] )
```

13. Weighted Avg Premium (Iterator)

Business Use: Average annual premium across all policies

```
Weighted_Avg_Premium = AVERAGEX( DIM_Policy, DIM_Policy[AnnualPremium] )
```

14. YTD Claims Amount (Time Intelligence)

Business Use: Cumulative claim amount from start of year to selected date

```
YTD_Claims_Amount = TOTALYTD( SUM(FACT_Claims[ClaimAmount]), DIM_Date[Date] )
```

15. MTD Claims Amount (Time Intelligence)

Business Use: Cumulative claim amount from start of month to selected date

```
MTD_Claims_Amount = TOTALMTD( SUM(FACT_Claims[ClaimAmount]), DIM_Date[Date] )
```

16. QTD Claims Amount (Time Intelligence)

Business Use: Cumulative claim amount from start of quarter

```
QTD_Claims_Amount = TOTALQTD( SUM(FACT_Claims[ClaimAmount]), DIM_Date[Date] )
```

17. Prev Year Claims (Time Intelligence)

Business Use: Total claim amount in the same period last year for YoY comparison

```
Prev_Year_Claims = CALCULATE( SUM(FACT_Claims[ClaimAmount]),  
SAMEPERIODLASTYEAR(DIM_Date[Date]) )
```

18. YoY Growth % (Time Intelligence)

Business Use: Year-over-year percentage growth in claims

```
YoY_Growth_Pct = DIVIDE( [Total_Claim_Amount] - [Prev_Year_Claims], [Prev_Year_Claims], 0  
) * 100
```

19. Rolling 3 Month Claims (Time Intelligence)

Business Use: Sum of claims in the last 3 months — trend indicator

```
Rolling_3M_Claims = CALCULATE( SUM(FACT_Claims[ClaimAmount]),  
DATESINPERIOD(DIM_Date[Date], LASTDATE(DIM_Date[Date]), -3, MONTH) )
```

20. Prev Month Claims (Time Intelligence)

Business Use: Claim amount in the immediately preceding month

```
Prev_Month_Claims = CALCULATE( SUM(FACT_Claims[ClaimAmount]),  
PREVIOUSMONTH(DIM_Date[Date]) )
```

21. Approved Claims Count (Filter)

Business Use: Count of approved claims with CALCULATE filter

```
Approved_Claims_Count = CALCULATE( COUNT(FACT_Claims[ClaimID]), FACT_Claims[ClaimStatus]  
= "Approved" )
```

22. High Value Claims (Filter)

Business Use: Count of claims above Rs. 2,00,000

```
High_Value_Claims = CALCULATE( COUNT(FACT_Claims[ClaimID]), FACT_Claims[ClaimCategory] =  
"High" )
```

23. Fraud Claims Amount (Filter)

Business Use: Total value of suspected fraudulent claims

```
Fraud_Claims_Amount = CALCULATE( SUM(FACT_Claims[ClaimAmount]), FACT_Claims[FraudFlag] =  
"Yes" )
```

24. Active Policy Premium (Filter)

Business Use: Premium revenue from active policies only

```
Active_Policy_Premium = CALCULATE( SUM(DIM_Policy[AnnualPremium]),  
DIM_Policy[PolicyStatus] = "Active" )
```

25. Health Policy Count (Filter)

Business Use: Number of health insurance policies sold

```
Health_Policy_Count = CALCULATE( COUNTROWS(DIM_Policy), DIM_Policy[PolicyType] = "Health"  
)
```

26. Claim Std Deviation (Statistical)

Business Use: Statistical spread of claim amounts — risk volatility indicator

```
Claim_Std_Dev = STDEV.P(FACT_Claims[ClaimAmount])
```

27. Median Claim Amount (Statistical)

Business Use: Median claim value — more robust than average for skewed data

```
Median_Claim_Amount = MEDIAN(FACT_Claims[ClaimAmount])
```

28. Top Agent by Premium (Ranking)

Business Use: Ranks agents by premium collected — use in agent leaderboard visual

```
Top_Agent_Premium_Rank = RANKX( ALL(DIM_Agent[AgentName]),  
CALCULATE(SUM(DIM_Policy[AnnualPremium])), , DESC, DENSE )
```

29. Top Policy Type Rank (Ranking)

Business Use: Ranks policy types by premium — helps identify top products

```
Policy_Type_Rank = RANKX( ALL(DIM_Policy[PolicyType]), [Total_Annual_Premium], , DESC, DENSE )
```

30. Claims Above Average Flag (Logical)

Business Use: Classifies claim as above or below average — used in conditional formatting

```
Claims_Above_Avg = IF( SUM(FACT_Claims[ClaimAmount]) > [Avg_Claim_Amount], "Above Average", "Below Average" )
```

31. Agent Target Status (Logical)

Business Use: Shows if agent has achieved annual premium target

```
Agent_Target_Status = IF( CALCULATE(SUM(DIM_Policy[AnnualPremium])) >= MAX(DIM_Agent[Target_Annual]), "Target Achieved ✓", "Target Pending ✗" )
```

32. Distinct Policy Types (Text / Count)

Business Use: Count of unique policy types in the filtered context

```
Distinct_Policy_Types = DISTINCTCOUNT(DIM_Policy[PolicyType])
```

33. Distinct Customers with Claims (Text / Count)

Business Use: Count of unique customers who have filed at least one claim

```
Distinct_Claimants = DISTINCTCOUNT(FACT_Claims[CustomerID])
```

34. All Time Claims Amount (All / Remove Filter)

Business Use: Total claims ignoring date filter — used for % of total calculations

```
All_Time_Claims = CALCULATE( SUM(FACT_Claims[ClaimAmount]), ALL(DIM_Date) )
```

35. % of Total Claims (All / Remove Filter)

Business Use: Each row's share of total claims — used in bar/table visuals

```
Pct_of_Total_Claims = DIVIDE( SUM(FACT_Claims[ClaimAmount]), [All_Time_Claims], 0 ) * 100
```

36. Customer Segment in Claims (Related / Inactive Rel)

Business Use: Uses USERELATIONSHIP for inactive relationship between Claims and Customer

```
Customer_Segment_via_Claim = CALCULATE( COUNT(FACT_Claims[ClaimID]), USERELATIONSHIP(FACT_Claims[CustomerID], DIM_Customer[CustomerID]) )
```

37. Top 5 Claim Amount (Table Function)

Business Use: Sum of top 5 highest claim amounts using TOPN + SUMX

```
Top5_Claim_Amount = SUMX( TOPN(5, FACT_Claims, FACT_Claims[ClaimAmount], DESC), FACT_Claims[ClaimAmount] )
```

38. Cumulative Claims (Table Function)

Business Use: Running total of claims over time — used in running total line chart

```
Cumulative_Claims = CALCULATE( COUNT(FACT_Claims[ClaimID]), FILTER( ALL(DIM_Date[Date]),  
DIM_Date[Date] <= MAX(DIM_Date[Date]) ) ) )
```

39. KPI Color Code (Switch / Multi)

Business Use: Returns color string for conditional KPI card coloring using SWITCH(TRUE())

```
Settlement_Color = SWITCH( TRUE(), [Claim_Settlement_Ratio] >= 90, "Green",  
[Claim_Settlement_Ratio] >= 75, "Yellow", "Red" )
```

40. Net Claims Liability (Variable)

Business Use: Uses VAR...RETURN pattern for clean calculation of outstanding claim liability

```
Net_Claims_Liability = VAR total_claimed = SUM(FACT_Claims[ClaimAmount]) VAR  
total_approved = SUM(FACT_Claims[ApprovedAmount]) VAR liability = total_claimed -  
total_approved RETURN IF(liability > 0, liability, 0)
```

6. Dashboard Design — 5 Pages

Page 1: Executive Overview

- KPI Cards: Total Policies, Total Premium, Total Claims Filed, Settlement Ratio, Loss Ratio
- Line Chart: Monthly Claims Amount (2019–2024) with trend line
- Donut Chart: Policy Type distribution by count
- Bar Chart: Claims by State (Top 10)
- Card: YTD vs PYTD Claims with % change indicator
- Slicer: Year, PolicyType (synced across pages)

Page 2: Claims Analysis

- KPI Cards: Total Claim Amount, Approved Amount, Rejection Rate, Avg Processing Days, Fraud Claims
- Clustered Bar: Claim Status breakdown by Policy Type
- Matrix: Rejection Reason vs Policy Type with % values
- Scatter Plot: Claim Amount vs Processing Days (by Claim Category)
- Line Chart: Monthly claims trend with Rolling 3M line
- Drill-through enabled: Click on ClaimID → Claim Detail Page

Page 3: Policy Performance

- KPI Cards: Active Policies, Lapsed Policies, Avg Annual Premium, Total Coverage Amount
- Stacked Bar: Premium by Policy Type and Payment Frequency
- Area Chart: Policies sold over time (by PolicyStartDate Year)
- Treemap: Policy distribution by Channel
- Table: Top 10 Policies by Premium with Policy Status indicator
- Slicer: PolicyStatus, PolicyType, Channel

Page 4: Agent Performance

- KPI Cards: Total Agents, Active Agents, Total Premium via Agents, Avg Target Achievement
- Leaderboard Table: Agent Name, Policies Sold, Total Premium, Rank, Target Status
- Bar Chart: Premium collected by Agent Region
- Scatter Plot: Experience Years vs Premium Collected
- Map Visual: Agent locations by State (bubble size = premium)
- Conditional Formatting: Red/Yellow/Green for Target Status

Page 5: Customer Insights

- KPI Cards: Total Customers, Premium Segment %, Avg Age, Avg Annual Income, Repeat Claimants
- Bar Chart: Customers by Segment (Premium/Standard/Basic)
- Stacked Bar: Claims by Gender and Policy Type

- Funnel: Customer Journey — Member → Policy → Claim → Approval
- Table: Customer 360 — CustomerID, Name, Policies Count, Total Claims, Approved Amount
- Slicer: State, CustomerSegment, Gender, MaritalStatus

7. Step-by-Step Power BI Setup Guide

Step 1: Import Data

- Open Power BI Desktop → Get Data → Excel Workbook
- Import all 4 files: 01_DIM_Customer.xlsx, 02_DIM_Policy.xlsx, 03_FACT_Claims.xlsx, 04_DIM_Agent.xlsx
- In each file, select the correct sheet (DIM_Customer, DIM_Policy, FACT_Claims, DIM_Agent)
- Click 'Transform Data' to open Power Query Editor

Step 2: Data Cleaning in Power Query

- Verify data types: Date columns → Date; Numeric columns → Whole Number or Decimal
- CustomerID, PolicyID, AgentID, ClaimID → Text type
- Check for nulls in key columns: CustomerID, PolicyID (should be 0 nulls)
- Rename queries to match table names (DIM_Customer, DIM_Policy, FACT_Claims, DIM_Agent)
- Click 'Close & Apply' to load data

Step 3: Create Date Table

- Go to Modeling tab → New Table
- Paste the CALENDAR() DAX formula from Section 4 of this document
- After table is created: Modeling → Mark as Date Table → select 'Date' column

Step 4: Build Relationships (Model View)

- Click 'Model' icon (left sidebar) to open Model View
- Drag FACT_Claims[CustomerID] → DIM_Customer[CustomerID] (Many to One, Single filter)
- Drag FACT_Claims[PolicyID] → DIM_Policy[PolicyID] (Many to One, Single filter, ACTIVE)
- Drag DIM_Policy[AgentID] → DIM_Agent[AgentID] (Many to One, Single filter)
- Drag FACT_Claims[ClaimDate] → DIM_Date[Date] (Many to One, Single filter)
- IMPORTANT: The DIM_Policy[CustomerID] → DIM_Customer[CustomerID] will auto-create — SET THIS AS INACTIVE
- Verify: Model should show clean Star Schema with no circular paths

Step 5: Create All 40 DAX Measures

- In FACT_Claims table → New Measure (right-click table name)
- Create a dedicated Measures Table: Modeling → New Table → Measures = ROW("x",1) then delete the column
- Paste each DAX measure from Section 5 one by one
- Organize measures into Display Folders: Select measure → Properties → Display Folder
- Folders: 'Claims KPIs', 'Premium KPIs', 'Time Intelligence', 'Ratios', 'Agent KPIs'

Step 6: Build Dashboard Pages

- Add 5 Report pages, name them as per Section 6
- Use Card visual for all KPI measures
- Enable Sync Slicers: View → Sync Slicers for Year and PolicyType slicers across pages
- Add drill-through: Right click ClaimID → Add as Drill-Through field in Page 2
- Add dynamic title: Title text → use DAX measure like: 'Claims Dashboard | ' & SELECTEDVALUE(DIM_Date[Year], "All Years")
- Format consistently: Company color #1F4E79 (dark blue) as primary brand color

Step 7: Publish & Interview Tips

- File → Publish → My Workspace (requires Power BI Pro account)
- In interview: Explain WHY Star Schema — performance, simpler DAX, filter propagation
- Explain CALCULATE as the most important DAX function — it modifies filter context
- Know the difference: Row Context vs Filter Context
- Explain USERELATIONSHIP for inactive relationships
- Discuss how Loss Ratio, Settlement Ratio are insurance industry KPIs

8. Interview Questions & Model Answers

These are the most common Power BI interview questions you will be asked based on this project. Study these carefully.

Q1. Why did you choose Star Schema for this project?

Star Schema is the recommended data model for Power BI because: (1) Dimension tables are denormalized making queries faster, (2) DAX measures are simpler as filters flow from dimensions to fact, (3) Relationships are clean with no circular dependencies, (4) Power BI's Vertipaq engine is optimized for this structure.

Q2. Explain CALCULATE() with an example from this project.

CALCULATE() changes the filter context of a measure. Example: `Approved_Claims_Count = CALCULATE(COUNT(FACT_Claims[ClaimID]), FACT_Claims[ClaimStatus] = "Approved")` — here CALCULATE overrides the default filter and adds a new condition for ClaimStatus.

Q3. What is the difference between Row Context and Filter Context?

Filter Context is created by slicers, visuals and CALCULATE() — it filters the table. Row Context is created by iterators like SUMX, AVERAGEX — it processes each row individually. Example: `SUMX(FACT_Claims, FACT_Claims[ClaimAmount])` iterates row by row (Row Context) and then sums (Filter Context applies outside).

Q4. Why did you use USERELATIONSHIP() in your project?

DIM_Policy has CustomerID which creates an ambiguous path to DIM_Customer (already connected via FACT_Claims). To avoid circular dependency, one relationship is kept INACTIVE. USERELATIONSHIP() in a CALCULATE statement temporarily activates this inactive relationship when needed.

Q5. How did you create the Date Table and why is it important?

Created using CALENDAR() DAX function with ADDCOLUMNS() to add Year, Month, Quarter etc. Marked it as Date Table. This is essential for Time Intelligence functions like TOTALYTD(), SAMEPERIODLASTYEAR(), PREVIOUSMONTH() to work correctly.

Q6. Explain the Loss Ratio KPI.

Loss Ratio = (Total Claims Paid / Total Premium Collected) x 100. A ratio below 70% means the company is profitable. Above 100% means the insurer is paying out more in claims than it earns in premiums — unsustainable.

Q7. What is RANKX() and how did you use it?

RANKX() ranks a table by a measure. Used to rank agents by premium collected: `RANKX(ALL(DIM_Agent[AgentName]), CALCULATE(SUM(DIM_Policy[AnnualPremium])),, DESC, DENSE)`. ALL() removes the filter so all agents are compared against each other.

Q8. What is the difference between SUM and SUMX?

SUM aggregates a single column directly: SUM(Table[Column]). SUMX is an iterator that evaluates an expression row-by-row and then sums: SUMX(Table, expression). Use SUMX when you need to calculate something per row before aggregating.

9. Business Requirements Document (BRD)

BR #	Stakeholder	Requirement	KPI / Visual	Priority
BR 01	CEO	Monitor overall portfolio health	Loss Ratio, Settlement Ratio — KPI Cards on Overview	High
BR 02	Claims Head	Reduce claim processing time	Avg Processing Days, Claims by Status — Bar Chart	High
BR 03	Sales Head	Track agent productivity vs target	Agent Leaderboard with Target Status	High
BR 04	Risk Team	Identify fraud-prone claim types	Fraud Flag analysis by ClaimType and Region	High
BR 05	Finance	Premium vs claims liability by product	Loss Ratio by PolicyType — Matrix	High
BR 06	Marketing	Understand customer segments	Segment distribution, income vs policy analysis	Medium
BR 07	Operations	Identify top rejection reasons	Rejection Reason Matrix, Pie Chart	Medium
BR 08	IT/Analytics	Year-over-year growth tracking	YoY Growth %, Prev Year Claims — Line Chart	Medium
BR 09	Regional Mgr	Performance by region/state	State-wise claims map, Agent region bar chart	Medium
BR 10	Compliance	Policy lapse rate monitoring	Lapsed Policy %, Surrendered policy count	Low

10. Project Summary & Files Checklist

Deliverable	File Name	Contents
Customer Data	01_DIM_Customer.xlsx	500 rows, 16 columns — Customer dimension
Policy Data	02_DIM_Policy.xlsx	800 rows, 17 columns — Policy dimension
Claims Data	03_FACT_Claims.xlsx	1,200 rows, 18 columns — Central fact table
Agent Data	04_DIM_Agent.xlsx	15 rows, 14 columns — Agent dimension
Date Table	Created inside Power BI	CALENDAR() DAX — 2019 to 2024
Documentation	PowerBI_Insurance_Project_Documentation.pdf	This file — full project guide

This project covers: Star Schema Modelling • Date Table with CALENDAR() • 40 unique DAX functions • 5 Dashboard Pages • Insurance Domain KPIs • Interview Preparation. All datasets are designed with clean primary-foreign key relationships to ensure zero relationship ambiguity in Power BI.